



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Отчет по выполнению практического задания №6.2

Тема: Поиск образца в тексте

Дисциплина: структуры и алгоритмы обработки данных

Выполнил студент
группа

Мижурин А.А.
ИКБО-50-25

Москва 2026

ЦЕЛЬ РАБОТЫ

Цель работы: изучить алгоритм Кнута–Морриса–Пратта и применить его для поиска образца в тексте, исследовать число сравнений при успешном и безуспешном поиске.

В работе используется **вариант 3**: требуется найти индекс последнего вхождения образца в текст.

1 ХОД РАБОТЫ

1.1 Формулировка задачи

Используя алгоритм Кнута–Морриса–Пратта, найти индекс последнего вхождения образца в текст. Задание выполняется по **варианту 3**.

Дополнительно требуется разобрать поиск на примере, подсчитать число сравнений при успешном обнаружении первого вхождения и при безуспешном поиске, определить предусловия и постусловия функций, а также проверить программу на небольших и больших наборах данных.

1.2 Математическая модель решения

1.2.1 Префикс-функция

Пусть дан образец P длины m . Для каждой его позиции строится префикс-функция π . Значение $\pi[i]$ равно длине наибольшего собственного префикса подстроки $P[0...i]$, который одновременно является её суффиксом.

Например, для образца АВА получается массив $[0, 0, 1]$: у строки АВА совпадают префикс и суффикс А длины 1. Этот массив показывает, с какой позиции образца можно продолжить поиск после несовпадения, не возвращаясь назад по уже просмотренной части текста.

1.2.2 Поиск последнего вхождения

Используются два индекса: `textIndex` указывает на символ текста, а `patternIndex` — на символ образца. Если символы совпали, оба индекса увеличиваются. При несовпадении индекс образца переносится по префикс-функции. Индекс текста при этом не уменьшается.

Когда совпал весь образец, запоминается индекс начала найденного вхождения. Поиск не завершается: индекс образца переносится по префикс-функции, после чего просмотр продолжается. Поэтому каждое следующее найденное вхождение заменяет предыдущее, и после просмотра всего текста

сохранён именно последний индекс. Такой переход также позволяет находить перекрывающиеся вхождения. Если ни одного совпадения нет, возвращается -1.

Счётчиком учитываются только сравнения символа текста с символом образца во время поиска. Сравнения, необходимые для предварительного построения префикс-функции, в этот счётчик не входят.

1.2.3 Разбор успешного поиска

Рассмотрим текст АВАВАВА и образец АВА. Первое вхождение обнаруживается после трёх сравнений:

Таблица 1.1 – Поиск первого вхождения образца АВА

№	Индекс текста	Индекс образца	Символы	Результат
1	0	0	A = A	Совпадение
2	1	1	B = B	Совпадение
3	2	2	A = A	Найден индекс 0

После первого результата алгоритм продолжает просмотр. Вхождения начинаются с индексов 0, 2 и 4. Последнее вхождение имеет индекс 4, а за полный поиск выполняется 7 сравнений.

1.2.4 Разбор безуспешного поиска

Для текста АВАВАВ и образца АВАС префикс-функция равна [0, 0, 1, 0]. После несовпадения В и С алгоритм возвращает индекс образца к значению 1 и повторно сравнивает тот же символ текста с В.

Таблица 1.2 – Безуспешный поиск образца АВАС

№	Индекс текста	Индекс образца	Символы	Результат
1	0	0	A = A	Совпадение
2	1	1	B = B	Совпадение
3	2	2	A = A	Совпадение
4	3	3	B ≠ C	Переход к позиции 1
5	3	1	B = B	Совпадение
6	4	2	A = A	Совпадение
7	5	3	B ≠ C	Переход к позиции 1
8	5	1	B = B	Текст закончился

Выполнено 8 сравнений, полное совпадение не получено, поэтому результат равен -1.

1.2.5 Функции, предусловия и постусловия

Функция `buildPrefixFunction` получает образец и возвращает массив той же длины. Предусловие: переданный объект `string_view` ссылается на существующую последовательность символов на всё время вызова. Постусловие: каждый элемент результата содержит длину наибольшей границы соответствующего префикса образца. Для пустого образца возвращается пустой массив.

Функция `findLastOccurrenceKmp` получает текст и образец. Предусловие: образец не пуст, а оба объекта `string_view` действительны на всё время вызова. При нарушении требования к образцу выбрасывается `invalid_argument`. Постусловие: поле `lastIndex` результата содержит индекс начала последнего вхождения либо `-1`, если вхождений нет; поле `comparisons` содержит число выполненных сравнений символов текста и образца. Исходные строки не изменяются.

1.2.6 Практическая сложность

Префикс-функция строится одним последовательным просмотром образца с возвратами только по уже вычисленным значениям. Во время поиска индекс текста не движется назад. Поэтому при увеличении текста объём работы растёт примерно пропорционально его длине, а увеличение образца прежде всего увеличивает объём предварительной обработки. Повторный просмотр уже проверенных фрагментов текста с их начала не выполняется.

Для практической оценки использованы строки из символов `a`. В образец после последовательности `a` добавлен символ `b`, которого нет в тексте. Это создаёт частые частичные совпадения и переходы по префикс-функции.

Таблица 1.3 – Зависимость числа сравнений от длины текста

Длина текста	Длина образца	Результат	Сравнений
10	5	-1	16
100	5	-1	196
1 000	5	-1	1 996
10 000	5	-1	19 996

Увеличение текста в десять раз приводит примерно к десятикратному увеличению числа сравнений. Следующая серия сохраняет длину текста, но изменяет длину образца.

Таблица 1.4 – Зависимость числа сравнений от длины образца

Длина текста	Длина образца	Результат	Сравнений
10 000	5	-1	19 996
10 000	10	-1	19 991
10 000	100	-1	19 901
10 000	1 000	-1	19 001

В выбранном безуспешном сценарии более длинный образец даёт более длинный начальный участок совпадения, но оставляет меньше позиций текста, на которых возникают несовпадение и повторное сравнение. Во всех измерениях число сравнений изменяется линейно, без многократного повторного просмотра текста.

1.3 Код программы с комментариями

Результат поиска объединяет последний индекс и счётчик сравнений. В объявлении функции зафиксированы её назначение, предусловие и постусловие.

Листинг 1.1 – Описание результата и функции поиска

```
struct SearchResult {
    std::ptrdiff_t lastIndex = -1;
    std::size_t comparisons = 0;
};

// Возвращает индекс последнего вхождения образца и число сравнений
// символов.
// Предусловие: образец не пуст.
// Постусловие: lastIndex равен индексу начала последнего вхождения либо
// -1,
// а comparisons содержит число сравнений символов текста и образца.
SearchResult findLastOccurrenceKmp(std::string_view text,
                                   std::string_view pattern);
```

Префикс-функция строится до начала поиска. При несовпадении значение `matched` уменьшается не на единицу, а сразу переводится к следующей возможной границе образца.

Листинг 1.2 – Построение префикс-функции

```
std::vector<std::size_t> buildPrefixFunction(std::string_view pattern) {
    std::vector<std::size_t> prefix(pattern.size(), 0);

    for (std::size_t i = 1; i < pattern.size(); ++i) {
        std::size_t matched = prefix[i - 1];

        while (matched > 0 && pattern[i] != pattern[matched]) {
            matched = prefix[matched - 1];
        }
        if (pattern[i] == pattern[matched]) {
            ++matched;
        }

        prefix[i] = matched;
    }

    return prefix;
}
```

Основная функция просматривает текст до конца. После полного совпадения она записывает найденный индекс и продолжает поиск, поэтому возвращает последнее, а не первое вхождение.

Листинг 1.3 – Поиск последнего вхождения алгоритмом КМП

```
SearchResult findLastOccurrenceKmp(std::string_view text,
                                   std::string_view pattern) {
    if (pattern.empty()) {
        throw std::invalid_argument("Образец не должен быть пустым");
    }

    const std::vector<std::size_t> prefix = buildPrefixFunction(pattern);
    SearchResult result;
    std::size_t textIndex = 0;
    std::size_t patternIndex = 0;

    while (textIndex < text.size()) {
        ++result.comparisons;

        if (text[textIndex] == pattern[patternIndex]) {
            ++textIndex;
            ++patternIndex;

            if (patternIndex == pattern.size()) {
                result.lastIndex =
                    static_cast<std::ptrdiff_t>(textIndex - pattern.size());
                // Продолжаем поиск и допускаем перекрывающиеся вхождения.
                patternIndex = prefix[patternIndex - 1];
            }
        } else if (patternIndex > 0) {
            // Символ текста сравнивается с более короткой границей.
            patternIndex = prefix[patternIndex - 1];
        } else {
            ++textIndex;
        }
    }
}
```

```

    return result;
}

```

В main предусмотрены обычный ввод, демонстрационный запуск и запуск автоматических тестов. Пользователь получает найденный индекс и фактическое число сравнений.

Листинг 1.4 – Вывод результата поиска

```

const task02::SearchResult result =
    task02::findLastOccurrenceKmp(text, pattern);

if (result.lastIndex == -1) {
    std::cout << "Образец в тексте не найден.\n";
} else {
    std::cout << "Индекс последнего вхождения: "
                << result.lastIndex << '\n';
}
std::cout << "Количество сравнений: " << result.comparisons << '\n';

```

1.4 Результаты тестирования

Автоматические тесты охватывают успешный и безуспешный поиск, перекрывающиеся вхождения, равенство текста и образца, образец длиннее текста, большие строки и нарушение предусловия.

Таблица 1.5 – Результаты тестирования алгоритма КМП

№	Текст	Образец	Ожидалось	Получено	Сравнений
1	ABABABA	ABA	4	4	7
2	algorithm	search	-1	-1	9
3	abc	abcd	-1	-1	3
4	aaaaa	aaa	2	2	5
5	KMP	KMP	0	0	3
6	100 000 a + b	aaaab	99 996	99 996	199 997
7	100 000 a	aaaab	-1	-1	199 996
8	abc	Пустой	Ошибка	Ошибка	—

Все десять автоматических проверок завершились успешно. В таблице одна строка может соответствовать нескольким проверкам: отдельно проверяются найденный индекс, число сравнений и факт выбрасывания исключения. Результаты малых и больших тестов совпали с ожидаемыми.

ВЫВОД

В работе реализован алгоритм Кнута–Морриса–Пратта для определения индекса последнего вхождения образца. Алгоритм корректно учитывает перекрывающиеся вхождения, сообщает об отсутствии образца и подсчитывает сравнения символов. На примерах установлено, что первое вхождение АВА в АВАВАВА обнаруживается за 3 сравнения, а безуспешный поиск АВАС в АВАВАВ выполняет 8 сравнений.

Практические измерения на текстах длиной до 100 000 символов показали линейный рост числа сравнений при увеличении входных данных. Все десять автоматических проверок выполнены успешно. Задача решена, цель работы достигнута.